2

MALWARE TRIAGE AND BEHAVIORAL ANALYSIS

In this chapter, you'll learn the basics of malware analysis, which, along with the next chapter, should give you a solid foundation for learning everything in the rest of this book. We'll begin by walking through the malware analysis process, starting with initial triage of suspicious files. Then, we'll dig into automated analysis in a sandbox environment, before wrapping up with a discussion of behavioral analysis in a virtual machine. As we progress through the chapter, I'll point out areas in the malware triage and behavioral-analysis process that are especially relevant to investigating evasive malware. I'll be focusing mostly on Windows executable files in this chapter and throughout this book.

As I noted in the introduction, this book assumes you already have at least a beginner's knowledge of malware analysis. This chapter therefore provides only the basic information required to quickly get you up to speed, and it skims over concepts that will be discussed in more detail in later chapters. I'll point you to those chapters where appropriate.

Let's get into it, starting with the importance of the analysis environment.

The Analysis Environment

Building a safe and effective analysis environment is critical for successful malware analysis. You should put some thought into your analysis environment and tailor it to your needs. Malware analysts and researchers often use virtual machines and sandboxes, which offer a controlled environment in which to monitor the malware's behavior. As a result, malware is increasingly using virtual machine and sandbox detection and circumvention techniques.

Before we go further, it's important to establish some definitions. A *virtual machine (VM)* emulates a physical computer but runs entirely within an application known as a *hypervisor*. The hypervisor provides a sort of container that allows safe execution of malicious code and safe detonation of malware. A *malware analysis sandbox* is typically (but not always) a type of VM that is configured to automatically analyze malware and produce a report or assessment of the malware's behaviors, capabilities, and properties. Some examples of sandboxes are the open source sandbox Cuckoo and the proprietary sandboxes Joe Sandbox and Any.Run. The main point here is that nearly all malware sandboxes are VMs, but not every VM is configured to be a malware sandbox.

A typical malware analysis lab environment often consists of a host system and one or more VMs and sandboxes. The host system stores and runs the analysis VMs and sandboxes, and it may have Windows, Linux, or macOS as well as a hypervisor installed. The operating system and software configured on the VMs and sandboxes depend on the type of malware the analyst is investigating. For Windows malware analysis, for example, the analyst might have VMs running Windows 7, Windows 10, and Windows 11, as well as many specialized malware analysis tools.

WARNING

If you're a beginner malware analyst, I highly recommend that you take a look at **Appendix A** *to get an idea of lab setup and safety before delving into malware analysis. Malware analysis carries risks, and it's important to limit them as much as possible.*

The Malware Analysis Process

Imagine that you're a malware analyst and you're given an unknown file to investigate. This file could have no additional context, or it could be part of a larger breach and an ongoing incident response investigation. Either way, you must answer the following questions:

- What type of file is this?
- When the file is opened, what does it do?
- Upon execution, what types of artifacts does the file create?
- Does the executed file attempt to connect to the internet or communicate on the local network? If so, to which IP addresses or domains?
- Does the executed file exhibit signs of potential malicious activity, such as hiding itself on the infected system, attempting to steal sensitive data, or attempting to detect malware analysis tools?
- If this file is malicious in nature, what are its capabilities and intentions?

These are questions that a good malware analysis process will help you answer. The exact process can differ from analyst to analyst, however. Expert analysts may deviate quite a bit from the many documented malware analysis processes, while beginner analysts might prefer to stick with a clear path. Most published malware analysis processes boil down to the same thing: start with the basics and slowly add in more advanced techniques as needed. For the remainder of this chapter, I'll discuss the first stages of malware analysis, or malware triage, followed by manual behavioral analysis. In the next chapter, I'll dive into the later stages of the malware analysis process.

Initial Malware Triage

The word *triage* originates from the field of medicine, where patients are assessed (triaged) when there aren't enough resources to treat all of them simultaneously. Patients with severe wounds are treated first, while those with minor scrapes and bruises can be treated later. Triaging malware is a similar concept. When faced with several different pieces of malware to investigate (during an incident, for example), an analyst must first triage

the files to get an initial assessment of their behaviors before choosing which sample to investigate first.

There are several objectives when it comes to initial triage. First, you need to determine what type of file you're dealing with. Is it a Microsoft Excel document? A PDF? A script? An executable? The answer informs the rest of the malware analysis process. Second, you need to obtain as much information about the file as possible. For example, is the file known to public malware repositories and other researchers? This will help drive the third objective, which is to determine whether the file is malicious, and if so, what class of malware it is. Ransomware? Infostealer? Finally, you should have a basic understanding of the file's capabilities. One of the primary objectives of initial triage is to help you determine your next steps in investigating the malware sample.

NOTE

In the following subsections, I'll walk you through the basic file triage steps for a threat investigation. If you wish to follow along, you can download the malware file from VirusTotal or MalShare using the following file hash:

SHA256:
8348b0756633b675ff22ed3b840497f2393e8d9587f8933ac2469d689c16368a

Identifying the File Type

One of the most basic but important steps of malware analysis is identifying the file type, which will inform how you'll approach your analysis, the tools you'll use, and the order of the steps you'll take. A file's type is signified by its *magic bytes* or *signature*, one or more bytes of data at the beginning of the file. You can view the magic bytes in a hex editor such as McAfee FileInsight. The file shown in **Figure 2-1** has the magic bytes `4D 5A` (`MZ` in ASCII), which is common for PE files.

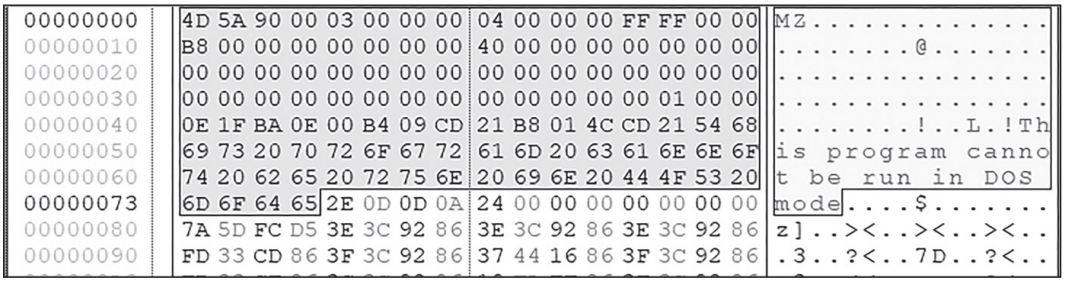

Figure 2-1: A PE header viewed in a hex editor

Table 2-1 lists some other common signatures, and you can find even more by searching "list of file signatures" on Wikipedia.

Table 2-1: Common File Signatures

Signature (ASCII)	Magic bytes	File type
`7z¼¯'`	`37 7A BC AF 27 1C`	7z archive
`ELF`	`7F 45 4C 46`	Executable and Linkable Format (ELF), an executable file type used in Unix- based systems
`%PDF-`	`25 50 44 46 2D`	PDF file
`{\rtf1`	`7B 5C 72 74 66 31`	Rich Text Format (RTF) document
`PK`	`50 4B 03 04`	ZIP file (and other files that use the *.zip* format, such as many Microsoft Office files)

In addition to a hex editor, you can use the `file` command to identify the file type in Linux. This tool reads the file's signature and displays it in a human-readable format. Simply run the `file` command with the malware file as an input parameter:

```
> file suspicious.exe
```

As you can see in the output shown here, this file is indeed an executable, specifically a Windows 32-bit PE file:

```
remnux@remnux:~$ file suspicious.exe
suspicious.exe: PE32 executable (GUI) Intel 80386, for MS Windows
```

The `file` command is a good general-purpose tool for identifying many common file formats, including non-PE files such as documents and archives. For PE files specifically, PE static analysis tools, or what I call *PE triage tools*, can come in handy. CFF Explorer (***https://ntcore.com***), for example, is a great initial analysis tool because it provides information such as file size and file creation timestamps, some of which you can see in **Figure 2-2**.

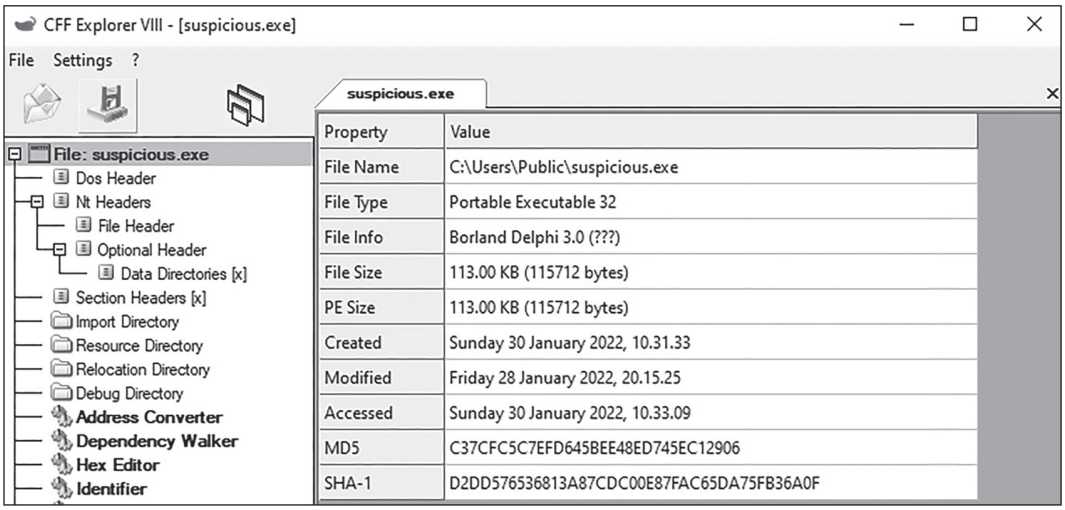

Figure 2-2: An executable file loaded into CFF Explorer

You may have noticed other information about the file in **Figure 2-2**, such as the Import Directory and Section Headers tabs on the left. I'll discuss more of these attributes later in this chapter and in the following chapter. Note the cryptic-looking MD5 and SHA-1 fields at the bottom right. These are file hashes, which we'll discuss next.

Obtaining the File's Hash

A file's *hash* is a sort of fingerprint in that it is unique to that file. When a file is run through a hashing algorithm, the algorithm generates a fixed-size sequence of characters. The exact size depends on which hashing algorithm is used. The most common file-hashing algorithms used for malware analysis are MD5, SHA-1, and (the most recent and reliable of the three) SHA256.

In **Figure 2-3**, the file's MD5 hash is `C37CFC5C7EFD645BEE48ED745EC12906` and its SHA-1 hash is `D2DD576536813A87CDC00E87FAC65DA75FB36A0F`. These hash values uniquely identify this file. Note that MD5 is an older algorithm, but it's still in use today. MD5 and SHA-1 have a risk of *collisions*, meaning that two or more files could have the same hash value; this is very rare, but it still happens. We won't go into hash collisions here; suffice to say, if you spot two completely different files with the same signature, you likely have encountered a collision.

Once you've obtained the file's hash, you can use it to get additional information about the file from other sources. Let's look at how that works.

Triaging with VirusTotal

VirusTotal (***https://www.virustotal.com***) is a publicly available platform for malware triage and analysis. No account is required, so anyone can upload files to get a quick assessment. VirusTotal runs the uploaded file against 60+ anti-malware software vendors to get the overall detection rate of the file, runs the file in a sandbox environment (which we'll discuss shortly), and retrieves additional information on the file from multiple sources. A typical VirusTotal assessment can include the following:

- The number of anti-malware detections for the file (the *detection rate*)
- Sandbox reports from the file
- The file's metadata (file creator, creation date, and so on)
- Digital certificates associated with the file
- Yara rule matches (we'll discuss Yara later in this chapter)
- Many other useful pieces of information

One key advantage of VirusTotal is its ability to query the very large malware database for a hash. You can simply paste the hash into VirusTotal, and it will run a passive query for the file, providing all the information as if you'd uploaded the file yourself. As long as the file is in the database, VirusTotal will provide a report on it. Running the SHA-1 hash for our malware file from **Figure 2-2** (`D2DD576536813A87CDC00E87FAC65DA75FB36A0F`) returns the report shown in **Figure 2-3**.

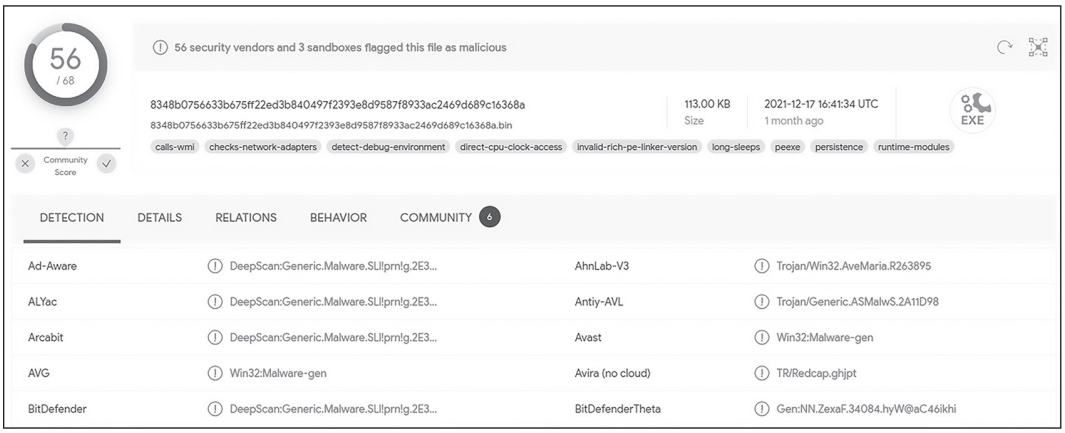

Figure 2-3: The VirusTotal report for the file from **Figure 2-2**

You can see that this file has a detection rate of 56/68, meaning that 56 out of 68 anti-malware software vendors classify this file as malicious. From this information, we can conclude that the file is highly likely to be malware. In addition to the file's detection rate, some vendors include the malware's class and family name. Based on the report, we can make an educated guess that the malware family is potentially *Ave Maria*, a common variant of remote access trojan and infostealer. Note, however, that the malware classifications from VirusTotal aren't always correct. The malware file may be packed, leading to a false classification. **Chapter 17** will discuss packing in great detail.

NOTE

Querying VirusTotal for a file hash is always a good idea. But before actually uploading a file there, you should consider the risks of the file being publicly available on the VirusTotal platform. Ask yourself: Does this file contain sensitive information, such as data about me or my company? Is this file part of an active investigation involving my company? Will uploading this file alert the malware authors that their malware was discovered

and is being actively analyzed? Remember, malicious actors are watching VirusTotal too.

Querying Search Engines and Other Resources

Along with VirusTotal, search engines can be powerful tools for malware triage. Simply paste the malware's hash or filename (if it is unique) into your search engine of choice and see what information is returned. If the file is already known, you may get valuable information about it from other online malware repositories and sandboxes.

Querying Google for our malware's SHA-1 hash returns the results shown in **Figure 2-4**.

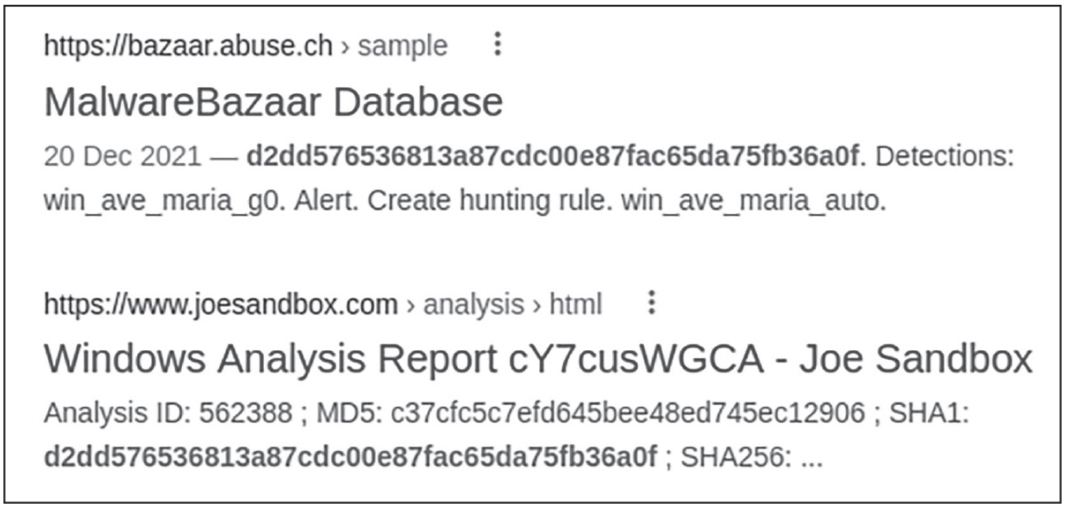

Figure 2-4: Querying our malware SHA-1 hash with Google

It seems this file is already quite well known! MalwareBazaar, a great resource for malware analysts and researchers, has some information on this file. Joe Sandbox (which I'll talk about later in this chapter) also seems to already know of it. Exploring these resources may help you better understand the file and its capabilities even before you analyze it yourself, saving you quite a bit of time and effort.

Now you should have at least a basic idea of what the file is and possibly even what malware family it belongs to, depending on whether it's available in public repositories. If the file is unknown, you'll need to dig

deeper to determine its capabilities and behaviors. But that's the fun part of malware analysis! Let's look at how to investigate an unknown file.

Identifying and Classifying Unknown Malware with Yara

Yara (***http://virustotal.github.io/yara/***) allows you to create signature definitions (called *rules*) designed to match on an unidentified file. These signature definitions can be in the form of strings, byte sequences, or other properties. The following code (available at ***https://github.com/bartblaze/Yara-rules/blob/master/rules/crimeware/AveMaria.yar***) shows an abridged version of a Yara rule written to detect Ave Maria:

```
rule AveMaria
{
  meta:
    --snip--
    source = "BARTBLAZE"
    author = "@bartblaze"
    description = "Identifies AveMaria aka WarZone RAT."
    category = "MALWARE"
    malware = "WARZONERAT"
    malware_type = "RAT"
    mitre_att = "S0534"

  strings:
    $ = "AVE_MARIA" ascii wide
    $ = "Ave_Maria Stealer OpenSource" ascii wide
    $ = "Hey I'm Admin" ascii wide
    $ = "WM_DISP" ascii wide fullword
    $ = "WM_DSP" ascii wide fullword
    $ = "warzone160" ascii wide

  condition:
    3 of them
}
```

This Yara rule is specifically designed to match on samples that may be related to the Ave Maria / Warzone RAT. It will match on any file that contains three or more of the strings in the `strings` section. Let's run this Yara

rule on our analysis sample. To run a Yara rule, use the following syntax (the `-s` parameter shows the exact string matches in the malware file):

```
$ yara -s rules_file malware_file
```

Running this Yara rule on our sample returns the following results:

```
remnux@remnux:/malware$ yara -s AveMaria.yar suspicious.exe

AveMaria 8348b0756633b675ff22ed3b840497f2393e8d9587f8933ac2469d689c16368a

0x162e0:$: A\x00v\x00e\x00_\x00M\x00a\x00r\x00i\x00a\x00 \x00S\x00t\x00e\x00a
x00l\x00e\x00r\x00 \x00O\x00p\x00e\x00n\x00S\x00o\x00u\x00r\x00c\x00e\x00
0x19340:$: H\x00e\x00y\x00 \x00I\x00'\x00m\x00 \x00A\x00d\x00m\x00i\x00n\x00
0x192b0:$: W\x00M\x00_\x00D\x00I\x00S\x00P\x00
0x19cca:$: W\x00M\x00_\x00D\x00I\x00S\x00P\x00
0x166c4:$: W\x00M\x00_\x00D\x00S\x00P\x00
0x1845a:$: W\x00M\x00_\x00D\x00S\x00P\x00
0x13c50:$: warzone160
```

It looks like we have a match! The line that begins with `AveMaria` shows us that there was a successful match, and the lines following it show exactly which strings from the rule matched on our suspicious file.

Yara rules can help you quickly obtain valuable information about the file you're dealing with and potentially even identify its associated malware family. For more information on Yara, see ***https://yara.readthedocs.io/en/stable/***.

Now, let's take a look at how to assess an unknown file based on its static properties.

Analyzing Static Properties

You can learn a lot about an unknown file by inspecting its static properties. Some of these properties were discussed in "The PE File Format" on **page 13**.

Strings

Strings are sequences of characters in various file types. Sometimes strings are human-readable text, and sometimes they're simply a sequence of bytes. Either way, they can be a great starting point for inspecting an unknown file. The simplest way to extract strings from a file is to use the `strings` command line tool in Linux. This tool scans the file and attempts to locate and interpret strings of binary data into human-readable form:

```
> strings suspicious.exe
```

Note that by default, the strings command will only output ASCII strings. Another type of string, unicode (or wide), can be extracted by using the following command: `strings -e l suspicious.exe`. The output already reveals some interesting things:

```
remnux@remnux:/malware$ strings suspicious.exe
--snip--
127.0.0.2
abcdefghijklmnopqrstuvwxyzABCDEFGHIJK...
warzone160
.bss
USER32.DLL
MessageBoxA
Assert
An assertion condition failed
PureCall
--snip--
```

Most notably, there's a reference to `warzone160`. Running a quick search engine query reveals that this is very likely related to the Ave Maria or Warzone RAT malware family, as you can see in **Figure 2-5**. This is a good example of integrating open source intelligence (OSINT) into your malware analysis process.

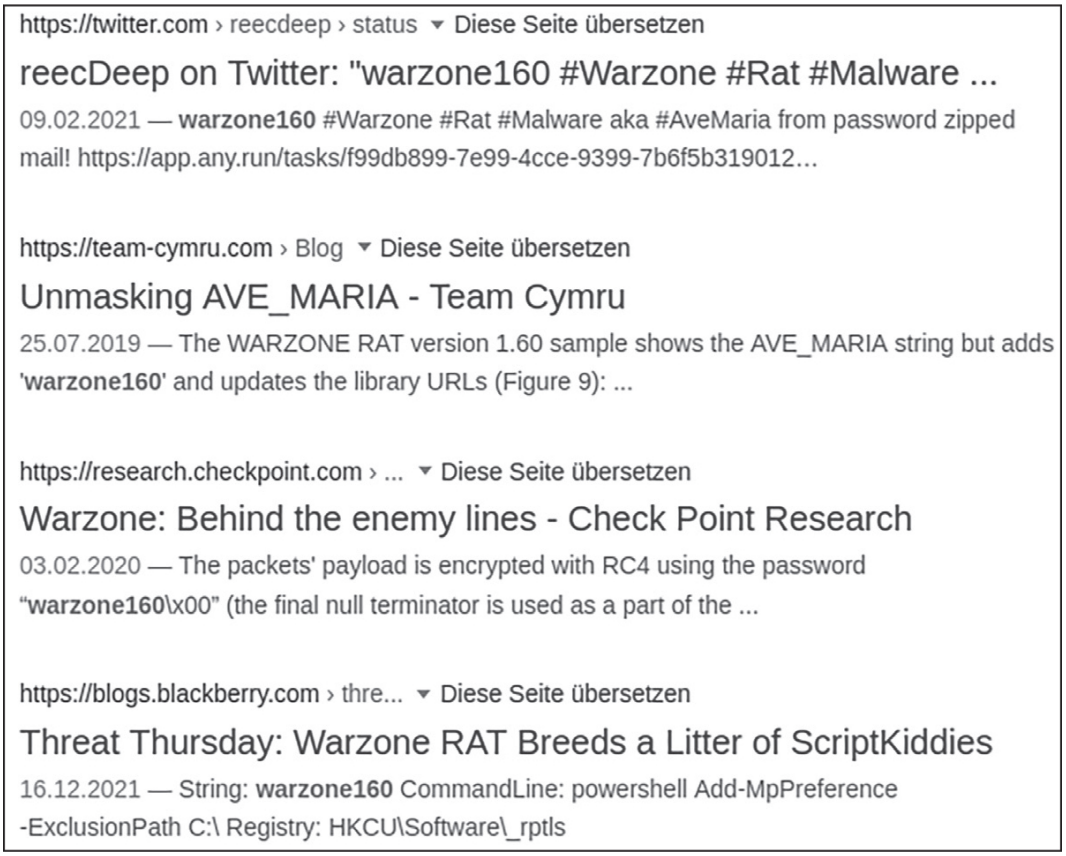

Figure 2-5: An OSINT investigation for embedded strings in malware

For executable files specifically, PE tools such as PEStudio can be very useful. PEStudio not only extracts various string formats from the executable but also orders and classifies those strings based on certain characteristics, as you can see in **Figure 2-6**.

encoding (2)	size (bytes)	file-offset	blacklist (100)	hint (235)	value (1279)
unicode	66	0x000191F0	-	wmi	Elevation:Administrator!new:{3ad05575-8857-4850-9277-11b85bdb8e09}
ascii	5	0x00013A74	-	utility	start
ascii	8	0x00014D70	-	utility	hostname
ascii	55	0x00016630	-	utility	cmd.exe /C ping 1.2.3.4 -n 2 -w 1000 > Nul & Del /f /q
ascii	12	0x00016770	-	utility	explorer.exe
ascii	43	0x00016828	-	utility	powershell Add-MpPreference -ExclusionPath
unicode	4	0x00013A88	-	utility	open
unicode	11	0x00014E18	-	utility	POP3 Server
unicode	9	0x00014E30	-	utility	POP3 User
unicode	11	0x00014E44	-	utility	SMTP Server
unicode	13	0x00014E5C	-	utility	POP3 Password
unicode	13	0x00014E78	-	utility	SMTP Password
unicode	11	0x00015CF0	-	utility	svchost.exe
unicode	14	0x00015D08	-	utility	svchost.exe -k
unicode	7	0x000160E4	-	utility	RDPClip
unicode	27	0x000165CC	-	utility	wmic process call create '"

Figure 2-6: String classification in PEStudio

PEStudio has discovered several notable strings: possible privilege escalation capabilities (`Elevation:Administrator`), a command line command (`cmd.exe`) and `powershell` and `wmic` references, as well as references to SMTP

services and passwords (`SMTP`, `POP`, and so on). From these strings, you might infer that this file has capabilities to elevate its privileges from a standard user to an administrator; invoke Windows tools such as *cmd.exe*, PowerShell, and WMIC; and use SMTP for network communication. The useful pieces of information you discover in strings can help guide you during your investigation, giving you clues about a malware file's intentions.

Two additional tools that are very useful for string analysis are FLOSS and StringSifter. FLOSS (***https://github.com/mandiant/flare-floss***) is a tool for identifying and extracting *obfuscated* strings—that is, strings that are being intentionally obscured to prevent prying eyes from viewing the data. Here's an excerpt of FLOSS's output for a different suspect file:

```
remnux@remnux:~$ floss unknown.exe
--snip--
FLOSS decoded 29 strings
--snip--
C:\Program Files (x86)\Microsoft\W0rd.exe
taskkill /f /im W0rd.exe
ZwQueryInformationProcess
--snip--
```

In this case, FLOSS was able to decode some notable obfuscated strings including a filepath (`C:\Program Files\Office\W0rd.exe`), a command line tool reference (`taskkill`), and what could be a Windows function that the malware will import at a later point (`ZwQueryInformationProcess`). In **Chapter 16**, I'll discuss methods with which malware can obfuscate data, and I'll also discuss how we can use FLOSS to reveal that data.

StringSifter (***https://github.com/mandiant/stringsifter***) takes the output from another string extraction tool, such as the aforementioned strings and FLOSS tools, and ranks and sorts the strings by their usefulness for and relevance to malware analysts. I won't discuss StringSifter more in this book, but it can be very helpful for quickly analyzing a large set of strings.

Imports and Exports

As **Chapter 1** explained, imports are libraries and functions that the executable file is using, while exports are functions that the executable provides to other functions or programs for their use. Imports and exports can be used to get hints about the executable file's intent. For example, if the file is importing libraries such as *Winhttp.dll*, we can make an educated guess (but always confirm!) that it may attempt to contact a remote server, such as a command and control server.

Once again, PEStudio can extract imports and exports from malware and display that information to you in an organized way, as demonstrated in **Figure 2-7**.

functions (204)	blacklist (85)	anonymous (14)	library (13)
BCryptSetProperty	x	-	bcrypt.dll
BCryptGenerateSymmetricKey	x	-	bcrypt.dll
BCryptOpenAlgorithmProvider	x	-	bcrypt.dll
BCryptDecrypt	x	-	bcrypt.dll
TerminateThread	x	-	kernel32.dll
WriteProcessMemory	x	-	kernel32.dll
OpenProcess	x	-	kernel32.dll
VirtualProtectEx	x	-	kernel32.dll
CreateRemoteThread	x	-	kernel32.dll
CreateProcessA	x	-	kernel32.dll
WriteFile	x	-	kernel32.dll

Figure 2-7: Function imports listed in PEStudio

Here we can see the various Windows functions that this malware is importing and is likely to call during its execution. Some of the functions of special interest are the `BCrypt*` functions (possibly used to encrypt or decrypt data), `WriteProcessMemory` and `CreateRemoteThread` (which may be used as part of process injection), and `WriteFile` (which is used to write data to a file). Although not definitive evidence of a file's maliciousness or capabilities, these functions are more clues that we can use during the malware analysis process.

Metadata and Other Information

Finally, the file's metadata can give us clues about its intentions. PEStudio can show the file's timestamps, which may represent when the file was first compiled. Also, it can show information about the programming language the file was written in, the various sections of the PE file, and even the embedded certificates that may have been used to sign the file. In short, you can gather a wealth of additional information about a file using PE file analysis tools such as PEStudio. Be aware, however, that malware authors can alter and fake metadata, as we'll explore later in this book.

Automated Malware Triage with Sandboxes

After initially assessing your suspect file, you likely will still have questions. Even if you were able to determine what malware family the sample belongs to based on your initial analysis, you may still need to quickly identify its capabilities and extract key information. A good option is to use a malware analysis sandbox, which can provide a lot of information about the sample's purpose, capabilities, and behaviors.

Malware analysis sandboxes are used to automate parts of the malware analysis process, especially initial triage. When a file is submitted to an automated sandbox, it is detonated (that is, executed) and its actions on the system are closely monitored. Automated sandboxes often produce a report of the file's behaviors and capabilities after analysis.

At the time of this writing, there are a myriad of sandboxes, each with different features. There are too many good sandboxes to list, but **Table 2-2** lists some I have experience with and feel are worth mentioning. Many of these sandboxes allow you to upload files free of cost, but some have a limit on the number of files you are able to submit or other limitations for their free tiers. Note that the first two items in the list (CAPE and Cuckoo) are not commercial, so you'll need to download the projects from GitHub and build them yourself. Additionally, take note that Cuckoo, at this time of writing, is not maintained anymore, but the project authors are working on a newer version.

Table 2-2: Commercial and Free Sandboxing Options

Name	Type	Source
CAPE	Free, open source	https://github.com/kevoreilly/CAPEv2https://capev2.readthedocs.io/
Cuckoo	Free, open source	https://github.com/cuckoosandbox
Hatching Triage	Commercial, free to submit files	https://tria.ge
Hybrid Analysis	Commercial, free to submit files	https://www.hybrid-analysis.com
Intezer	Commercial, free to submit files	https://www.intezer.com
Joe Sandbox	Commercial, free to submit files	https://www.joesecurity.org/
UnpacMe	Commercial, free to submit files	https://www.unpac.me/
VirusTotal	Commercial, free to submit files	https://www.virustotal.com
VMRay	Commercial	https://www.vmray.com

As with VirusTotal submissions, anything you submit to a public sandbox is immediately available to other researchers and the world. Make every effort to ensure the file you're submitting doesn't contain sensitive personal or business information, and consider the impact on the investigation if the sample is made available to the public.

Figure 2-8 shows the result of submitting our malware file to a Cuckoo sandbox instance.

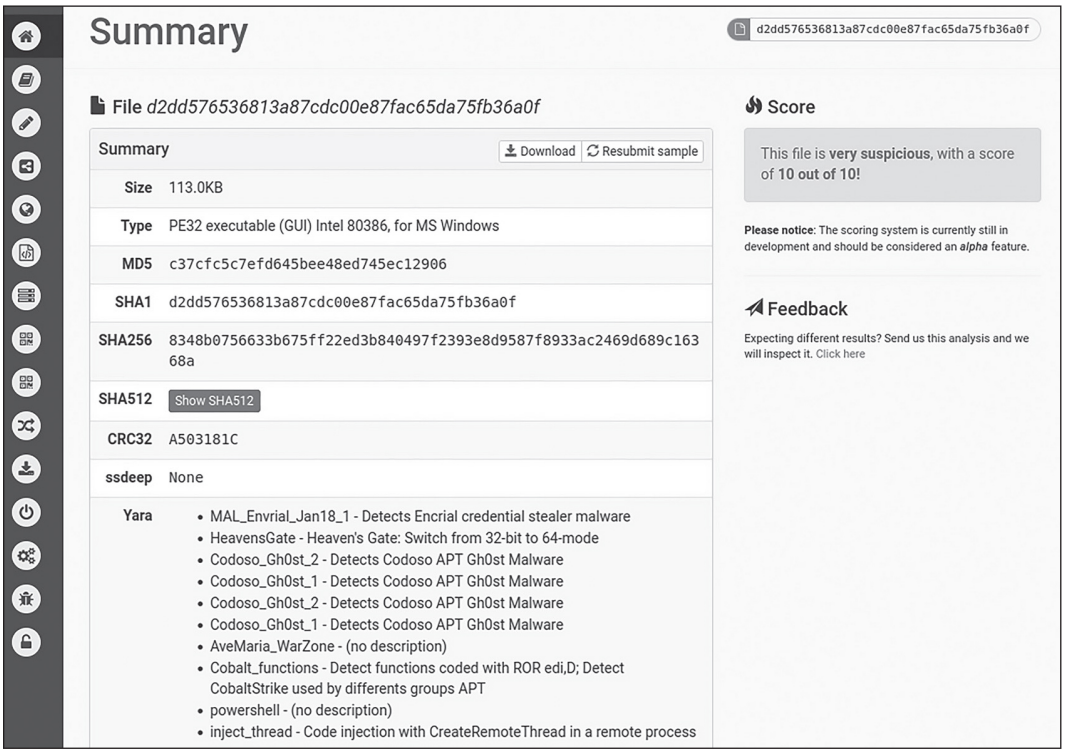

Figure 2-8: A malware summary in the Cuckoo sandbox

This Cuckoo summary page has quite a bit of useful information, such as basic file information (file type, size, hashes, and so on), the detection score ("10 out of 10!"), and even Yara signatures. It seems that Cuckoo's Yara engine is detecting this sample as possibly Gh0st or Ave Maria / Warzone. In comparison, Joe Sandbox reports that this sample is Ave Maria, as you can see in **Figure 2-9**.

Figure 2-9: A malware overview in Joe Sandbox

Simply knowing that this malware is likely associated with Ave Maria is helpful. This example also demonstrates that detonating a sample in more than one sandbox environment is never a bad idea. Sometimes different sandboxes can give different results, creating a more complete picture of the malware. Let's inspect this sample in a bit more depth in Joe Sandbox. The malware's process tree, shown in **Figure 2-10**, is a fundamental piece of information when you're analyzing malware samples in a sandbox.

Figure 2-10: The malware's process tree in Joe Sandbox

This process tree shows the original malware executable's process (*cY7cusWGCA.exe*) as well as all spawned child processes, which gives us some insight into the malware's capabilities and behaviors. Notably, the PowerShell process is executing the command `Add-MpPreference -ExclusionPath C:\`. This command adds the malware to the Windows Defender exclusion list, effectively bypassing anti-malware controls.

Also, the Anti Debugging section in Joe Sandbox, shown in **Figure 2-11**, illustrates some of the techniques that this sample may be using to detect and defend against debuggers.

Figure 2-11: The malware's anti-debugging techniques in Joe Sandbox

It appears that this malware sample may be using techniques such as dynamic library loading, manually reading the PEB, and calling `GetProcessHeap` to detect debuggers.

You can also see that Joe Sandbox has detected the use of several potential host defense evasion techniques, such as process injection and adding exclusions to Windows Defender (see **Figure 2-12**).

Figure 2-12: Defense evasion techniques identified in Joe Sandbox

Anti-debugging and defense evasion techniques will be discussed in greater detail in **Chapter 10** and **Part IV**, respectively.

NOTE

Using sandboxes is always a wise first step in identifying and locating malware evasion techniques so that you can later circumvent them if necessary. Keep in mind, however, that sandbox results may be inconclusive or even incorrect. Always manually investigate sandbox results to verify the findings.

Finally, employing sandboxes is a great way to quickly extract *indicators of compromise (IOCs)* from malware samples. IOCs can be network artifacts (such as communication to a specific domain or IP address, or a specific HTTP header) or host artifacts (such as a specific filename or registry key modification, or suspicious command line execution) that you can use later to detect this malware and prevent it from infecting further hosts. For more information on what is and isn't an IOC, visit ***https://www.crowdstrike.com/cybersecurity-101/indicators-of-compromise/***.

The Joe Sandbox environment was able to extract the command and control IP address from this sample, as shown in **Figure 2-13**.

Figure 2-13: A malware configuration extracted by Joe Sandbox

After analyzing the malware sample in a few sandboxes, we now have a fairly good assessment of many of its capabilities and behaviors. This malware sample is likely a variant of Ave Maria, and it is able to communicate with a command and control address (making it likely to download additional payloads), detect analysis tools such as debuggers, and evade host defenses using process injection and anti-malware bypass techniques. Depending on your malware analysis objectives for this investigation, this may be enough information for you. However, to understand a malware sample in detail, we must dive deeper.

Interactive Behavioral Analysis

Sandbox results can provide most of the information an analyst needs to determine a given malware's intent, purpose, and potential impact, but key questions will likely remain unanswered. Many malware sandboxes are designed for quick-and-dirty analysis and initial assessment, and sometimes they don't contain the detailed information that you may need during the investigation. Also, sandboxes can't be easily modified on the fly to tailor the environment to the running malware, so their results may be incomplete, especially if the malware is using advanced sandbox detection and circumvention techniques.

Interactive behavioral analysis is a fancy term for manually detonating and monitoring malware in a controlled environment, rather than relying solely on fully automated detonation in a sandbox such as Cuckoo. Interactive behavioral analysis is a much more manual and, well, interactive process, giving you more freedom. This interactive analysis is typically conducted in a VM.

One of the key reasons to use interactive behavioral analysis is that, if the malware sample is using sandbox detection techniques, you can attempt to thwart them by emulating a real user or otherwise giving the malware something it's looking for. For example, malware may be searching for a specific file on the victim's system, and since we fully control the interactive environment, we can provide this file so that the malware can continue executing. Fully automated sandboxes often fail in this regard.

The tools we'll explore are freely available. If you wish to follow along, you'll need to download and install the following tools in a VM environment:

- Procmon (***https://learn.microsoft.com/en-us/sysinternals/***)
- Process Hacker (***https://processhacker.sourceforge.io***)
- Fiddler (***https://www.telerik.com/fiddler***)
- Wireshark (***https://www.wireshark.org***)

NOTE

For this analysis example, we'll be working with a malware sample that you can find on VirusTotal or MalShare using the following hash:

SHA256: 9bbc55f519b5c2bd5f57c0e081a60d079b44243841bf0bc76eadf50a902aaa61

Monitoring Malware Behaviors

A large part of interactive behavioral analysis is monitoring the malware's behaviors or actions on the victim host. A popular tool for this is Process Monitor (Procmon), which is part of the Sysinternals suite. Procmon can capture many details of actions the malware takes on the host, such as spawning processes, reading and writing files and registries, and attempting to connect to the network. **Figure 2-14** shows the process tree in Procmon of a suspicious Microsoft Word file.

Figure 2-14: Analyzing a malware sample's process tree in Procmon

You can see that *WINWORD.EXE* (Microsoft Word) is spawning a suspicious process (*rundll32.exe*), a result that is always worth investigating further. The process tree can also be used to analyze parent-child process relationships and spot code injection mechanisms. We'll discuss code injection in **Chapter 12**.

By inspecting the Files section of Procmon, we can see that this file is creating some suspect additional files (see **Figure 2-15**).

Figure 2-15: Viewing suspicious file writes in Procmon

The main thing to notice here is the `WriteFile` operations, followed by the path where the file is being written. These two files (*diplo.ioe* and *flex.xz*) should be further inspected during analysis.

Similarly, **Figure 2-16** shows Procmon's Registry tab.

Figure 2-16: Viewing registry queries in Procmon

The *rundll32.exe* process shown here is executing a `RegQueryValue` operation, which indicates that it is reading several registry values on the host. The registry keys it's particularly interested in all seem to be related to domain, hostname, and network adapter information. Registry-reading operations are not necessarily malicious, as all Windows applications must read different hives in the registry for normal operation, but sometimes they can hint at what the malware is trying to accomplish. In the case of targeted and evasive malware, as you'll see throughout this book, it may be enumerating the registry and filesystem and searching for a specific value or pattern, such as a specific hostname or filepath.

Procmon is always a good first step in interactive behavior analysis. Suspicious activities revealed in Procmon can help further guide your investigation. For example, if the malware is writing to a strange file or reading a suspect registry key, part of interactive analysis is investigating those paths on the fly!

Another tool for interactive analysis is Process Hacker, which, along with similar tools such as Process Explorer, can be used to inspect the process tree, investigate the malware process's memory, and more. Memory inspection is a particularly useful task; you can do it by right-clicking the target process, selecting **Properties**, and then selecting the **Memory** tab. You can even query memory for specific string patterns. Searching for `http` is always a good start, as you can see in **Figure 2-17**.

Figure 2-17: Querying process memory for a string pattern in Process Hacker

The memory strings shown here contain some suspect data. We can see several URLs for domains such as *armerinin.com* and *siguages.ru*, which should be further investigated. The malware could be using these domains for command and control or downloading additional malware. Let's test that theory.

Inspecting Malware Network Traffic

Many malware samples will at some point attempt to connect to a remote server on the internet. They may do this for a variety of reasons, including the following:

- To download additional malicious files, payloads, and modules
- To communicate with a command and control server, requesting further instructions
- To send stolen information, such as credentials or files, to a remote server (often called *exfiltration*)
- To determine whether the infected host is currently connected to the internet or to get the host's public IP address (often used as a sandbox detection and evasion technique)

No matter the reason, it's important to identify when and to whom the malware is connecting. A *web proxy* is a type of tool that can intercept and manipulate network traffic being sent to and from the host, and it serves as a great malware analysis tool. The web proxy Fiddler, shown in **Figure 2-18**, has captured some suspicious malware internet connection attempts.

Figure 2-18: Malware internet connection attempts in the Fiddler web proxy

You can see that Fiddler has intercepted web requests to *api.ipify.org*, as well as three additional sites (*armerinin.com*, *houniant.ru*, and *siguages.ru*). If you query VirusTotal for the latter three domains, you'll probably see that they're rated as malicious. (They are at the time of this

writing, anyway.) The malware is likely attempting to download additional malware from one of these domains. The *api.ipify.org* site simply returns a host's external, public IP address. Why would the malware want to contact it? One possibility is that the malware is actually trying to obtain the host's public IP address to identify its hosting country. Another possible reason is to determine whether the host is online at all. Both pieces of information can be used for anti-analysis and evasion, and I'll discuss the associated techniques in greater depth throughout the book.

The popular network monitoring tool Wireshark can also be used to capture internet connection activity. Additionally, Wireshark can capture general network activity, such as traffic not destined for the internet, like host-to-host communication on a local network. **Figure 2-19** displays some of this malware sample's network connectivity in Wireshark.

Figure 2-19: Malware DNS requests captured in Wireshark

You can easily spot the DNS requests to the same hostnames that Fiddler found.

It's helpful to inspect HTTP and other protocol traffic in more detail. To view the web traffic, right-click an HTTP request and select **Follow ▸ TCP Stream**. You should see the HTTP POST request originating from this malware sample, as shown in **Figure 2-20**.

Figure 2-20: An HTTP POST request from the malware

This sample is sending data such as the victim's hostname, the IP address, and a unique identifier to its infrastructure (*siguages.ru*). Interestingly, the malware is also sending its *botnet ID* (in this case, "2209 _ubm"), which is an identifier assigned to a network of compromised computers. While this malware sends its data unencrypted, malware may also use encrypted channels for communication between the victim and its command and control infrastructure. This makes inspecting traffic in a web proxy or tool such as Wireshark more challenging. As we continue

through the book, we'll take a closer look at some of these techniques and how to overcome them.

NOTE

Many malware families try to determine whether a host is connected to the internet before infecting it as a sandbox evasion technique. If you're investigating the malware in an offline (non-internet-connected) VM, you'll likely want to use a tool that "fakes" network services, such as FakeNet or INetSim. I discuss these tools briefly in **Appendix A**.

Summary

This chapter covered the basics of triaging a malware sample to get a quick assessment and determine the next steps of your analysis. We discussed how automated malware sandboxes can be used as part of this process, as well as how to manually investigate malware behaviors in a controlled VM environment to understand them in greater detail. In the next chapter, we'll look at how code analysis can supplement these triage and behavioral-analysis techniques.